TASK 8 – Read and Print Data Safely

1. Objective

To design a safe input and output system for the Metro Gate Program using C++. The program validates user input, handles invalid entries without crashing, and displays a properly formatted journey report.

2. Safe Input Handling Using cin

The extraction operator (cin >>) is used to read numerical values such as card balance and journey fare.

- cin.fail() checks whether the input operation has failed, such as when the user enters letters instead of numbers.
- cin.clear() resets the error state of the input stream.
- cin.ignore() discards the remaining invalid characters from the input buffer.

This allows the program to recover from invalid input and request the value again instead of terminating unexpectedly.

3. Formatted Output Using <iomanip>

The <iomanip> library provides formatting tools to produce professional output.

Manipulator	**Purpose**
setw()	Sets the minimum field width.
setfill()	Fills unused spaces with a specified character.

fixed	Displays floating-point values in fixed-point notation.
setprecision(2)	Displays money with two decimal places.

These manipulators make financial information consistent and readable.

4. Neat Journey Report

The program generates a journey report containing:

- Card balance before travel.
- Journey fare.
- Remaining balance.
- Gate access status (APPROVED / DENIED).

The gate opens only when the card has sufficient balance.

5. Alternative Concept – getline() and stoi()

An alternative approach is to read the entire input line using std::getline() and convert it using std::stoi() inside a try-catch block.

For monetary values containing decimals, std::stod() should be used instead of std::stoi().

Example:

```
try {
   string input;
   getline(cin, input);

   double amount = stod(input);
}
```

```
catch (const invalid_argument&) {
    cout << "Invalid input!";
}
catch (const out_of_range&) {
    cout << "Value out of range!";
}
```

6. Complexity Analysis

Let L represent the number of characters in an input line.

Parameter	**cin >>**	**getline() + conversion**
Reading time	O(L)	O(L)
Conversion time	Integrated with extraction	O(L)
Auxiliary buffer space	O(1) for application-level storage	O(L)
Invalid input recovery	cin.clear() and cin.ignore()	try-catch
Input handling	Reads individual values	Reads the complete line

Evaluation:

Both approaches require O(L) time to process an input line. However, cin >> is convenient for reading individual numerical values and uses less application-level storage. The getline()

approach provides greater control over complete input lines and allows conversion errors to be handled through exceptions, but requires additional memory to store the line.

7. Conclusion

Safe input validation and formatted output improve the reliability and usability of the Metro Smart Card System. By handling invalid entries, preventing incorrect transactions, and presenting a clear journey report, the program provides a more dependable metro gate experience.